MAT700 Mathematical Methods for Data Mining - Lecture5

A. Balinsky

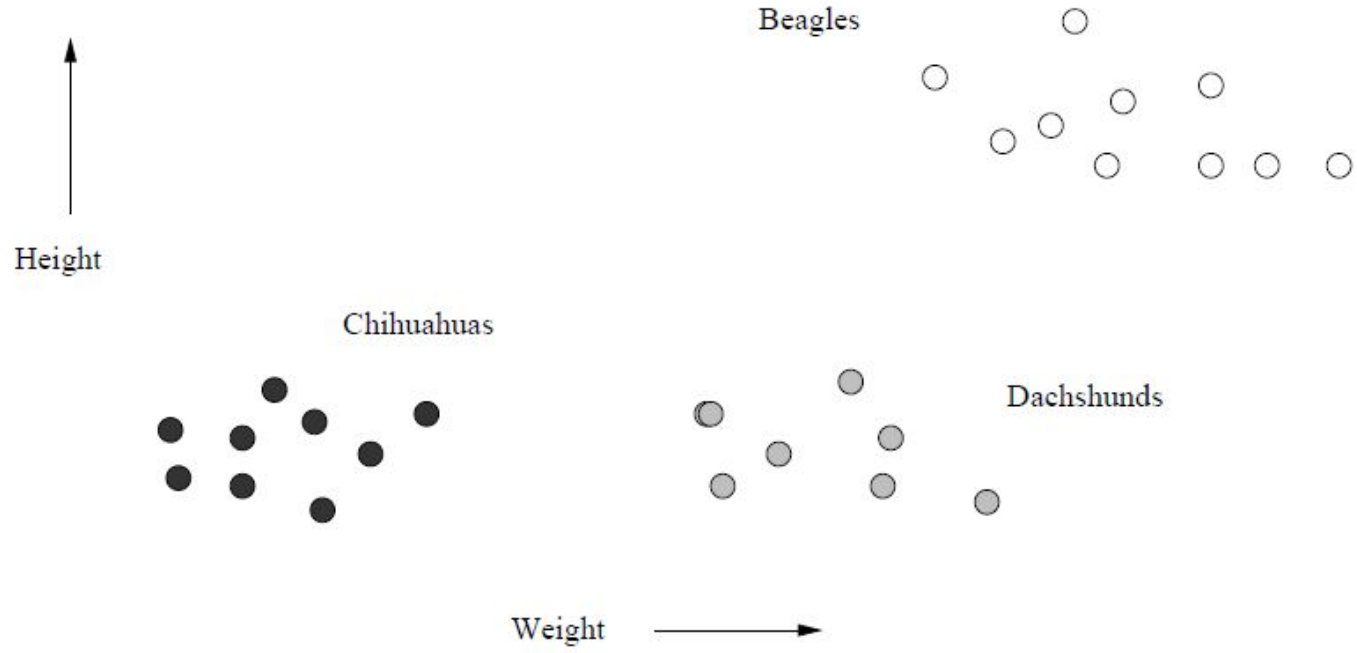
Cardiff School of Mathematics

Clustering

Clustering is the process of examining a collection of "points," and **grouping** the points into **"clusters"** according to some *distance measure*.

The **goal** is that points in the same cluster have a small distance from one another, while points in different clusters are at a large distance from one another.

Example



shows height and weight measurements of dogs of several varieties. Without knowing which dog is of which variety, we can see just by looking at the diagram that the dogs fall into three clusters, and those clusters happen to correspond to three varieties.

Our goal in this section is to offer methods for discovering clusters in data.

We are particularly interested in situations where the data is

- *very large*,
- and/or where the space either is *high-dimensional*,
- or the space is *not Euclidean* at all.

We begin with the basics:

- the two general approaches to clustering and
- the methods for dealing with clusters in a non- Euclidean space.

Clustering Strategies

We can divide clustering algorithms into two groups that follow two fundamentally different strategies.

Hierarchical or agglomerative algorithms

Hierarchical or agglomerative algorithms start with each point in its own cluster. Clusters are combined based on their "closeness", using one of many possible definitions of "**close**".

Combination stops when further combination leads to clusters that are undesirable for one of several reasons.

For example, we may stop when we have a *predetermined number* of clusters, or we may use a measure of *compactness* for clusters, and refuse to construct a cluster by combining two smaller clusters if the resulting cluster has points that are spread out over too large a region.

Point Assignment

The other class of algorithms involve ***point assignment***.

Points are considered in some order, and each one is assigned to the cluster into which it best fits.

This process is normally preceded by a short phase in which initial clusters are estimated.

Variations allow occasional combining or splitting of clusters, or may allow points to be unassigned if they are outliers (points too far from any of the current clusters).

Algorithms for clustering can also be distinguished by whether the algorithm assumes

- a **Euclidean** space,
- or whether the algorithm works for an **arbitrary distance measure**.

We shall see that a key distinction is that in a Euclidean space it is possible to summarize a collection of points by their **centroid** – the average of the points.

In a non-Euclidean space, there is no notion of a centroid, and we are forced to develop another way to summarize clusters.

Hierarchical Clustering

Any *hierarchical clustering* algorithm works as follows.

- We begin with every point in its own cluster.
- As time goes on, larger clusters will be constructed by combining two smaller clusters, and we have to decide in advance:
  - How will clusters be represented?
  - How will we choose which two clusters to merge?
  - When will we stop combining clusters?

Once we have answers to these questions, the algorithm can be described succinctly as:

```
WHILE it is not time to stop DO
    pick the best two clusters to merge;
    combine those two clusters into one cluster;
END;
```

Hierarchical Clustering in a Euclidean Space

To begin, we shall assume the space is Euclidean.

That allows us to **represent a cluster by its centroid** or average of the points in the cluster.

Note that in a cluster of one point, that point is the centroid, so we can initialize the clusters straightforwardly.

We can then use the merging rule that the distance between any two clusters is the Euclidean distance between their centroids, and we should pick the two clusters at the shortest distance.

Alternative Rules for Controlling Hierarchical Clustering

We have seen one rule for picking the best clusters to merge: find the pair with the smallest distance between their centroids.

Other ways to define inter-cluster distance are possible, and we can also pick the best pair of clusters on a basis other than their distance.

Some other options are:

- Take the distance between two clusters to be the minimum of the distances between any two points, one chosen from each cluster.

- Take the distance between two clusters to be the average distance of all pairs of points, one from each cluster.
- The **radius** of a cluster is the maximum distance between all the points and the centroid. Combine the two clusters whose resulting cluster has the lowest radius.
- The **diameter** of a cluster is the maximum distance between any two points of the cluster. We may choose to merge those clusters whose resulting cluster has the smallest diameter.

Hierarchical Clustering in Non-Euclidean Spaces

Given that we cannot combine points in a cluster when the space is non-Euclidean, our only choice is to pick one of the points of the cluster itself to represent the cluster. Ideally, this point is close to all the points of the cluster, so it in some sense lies in the "**center**".

We call the representative point the ***clustroid***.

We can select the *clustroid* in various ways, each designed to, in some sense, minimize the distances between the clustroid and the other points in the cluster.

Common choices include selecting as the clustroid the point that minimizes:

- The sum of the distances to the other points in the cluster.
- The maximum distance to another point in the cluster.
- The sum of the squares of the distances to the other points in the cluster.

K-means Algorithm

In this section we begin the study of **point-assignment algorithms**.

The best known family of clustering algorithms of this type is called **k-means**.

They assume a *Euclidean space*, and they also assume the number of clusters, k, is known in advance. It is, however, possible to deduce k by trial and error.

Partitioning Algorithm

Formally, given a data set, D, of n objects, and k, the number of clusters to form, a **partitioning algorithm** organizes the objects into k partitions $K \leq n$, where each partition represents a cluster. The clusters are formed to optimize an **objective partitioning criterion**.

k-Means: A Centroid-Based Technique

- Suppose a data set, D, contains n objects in Euclidean space. Partitioning methods distribute the objects in D into k clusters, $C_1, C_2, \ldots, C_k$, that is, $C_i \subset D$ and $C_i \cap C_j = \emptyset$ for $i \neq j$.

- An **objective function** is used to assess the partitioning quality so that objects within a cluster are similar to one another but dissimilar to objects in other clusters.

- A centroid-based partitioning technique uses the **centroid** of a cluster, C_i, to represent that cluster.
Conceptually, the centroid of a cluster is its center point.

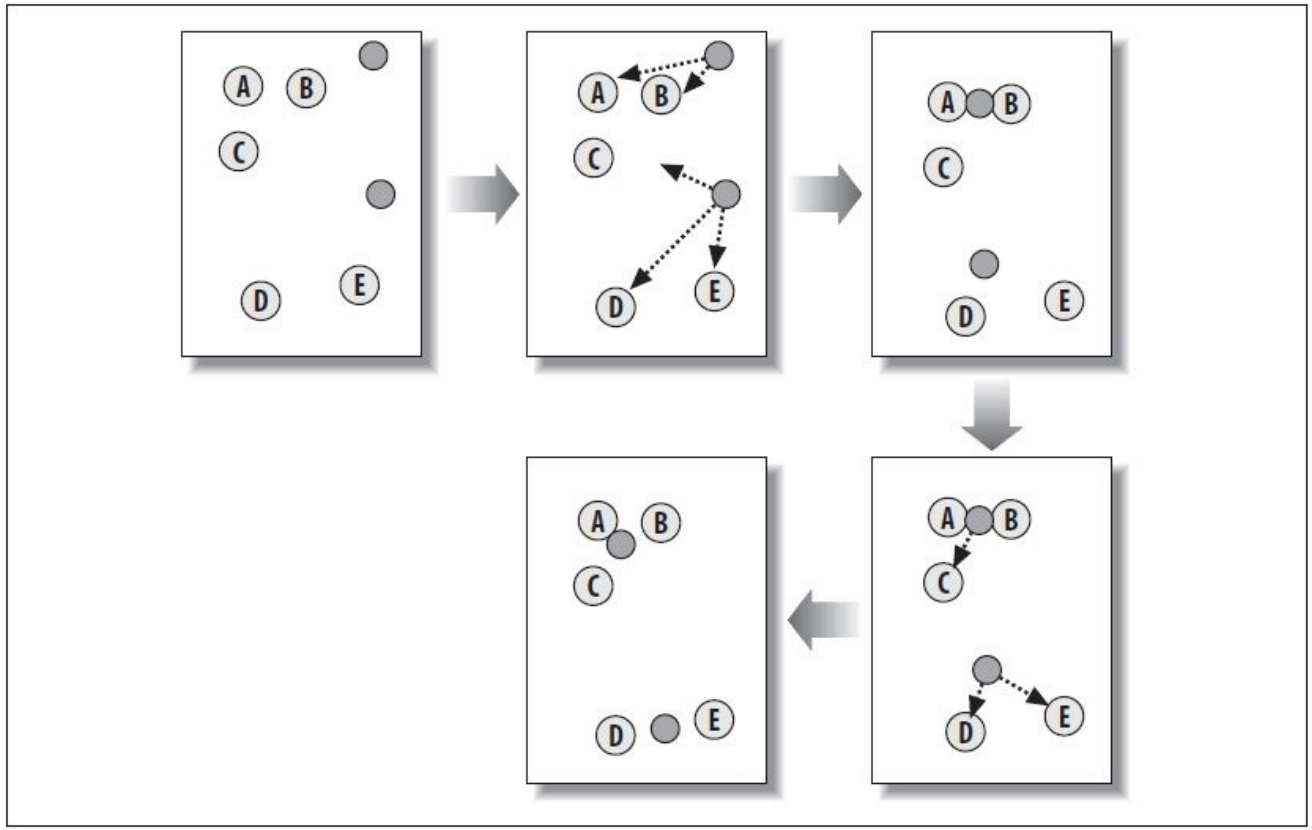
- **Objective Function**: The within-cluster variation, which is the sum of *squared error* between all objects in C_i and the centroid c_i, defined as

$$E = \sum_{i=1}^{k} \sum_{p \in C_i} dist(p, c_i)^2.$$

- Optimizing the within-cluster variation is computationally challenging.
  In the worst case, we would have to enumerate a number of possible partitionings that are exponential to number of clusters, and check the within-cluster variation values.
- It has been shown that the problem is NP-hard in general Euclidean space even for two clusters (i.e., $k = 2$).
  Moreover, the problem is NP-hard for general number of clusters even in 2-D Euclidean space.

k-Means: Greedy Approach

To overcome the prohibitive computational cost for the exact solution, greedy approaches are often used in practice. A prime example is the *k*-**means algorithm**, which us simple and commonly used.



K-means clustering with two clusters

k-Means

The k-Means Clustering Algorithm

1. Choose a value of k.
2. Select k objects in an arbitrary fashion. Use these as the *initial set* of k *centroids*.
3. Assign each of the objects to the cluster for which it is nearest to the centroid.
4. Recalculate the centroids of the k clusters.
5. Repeat steps 3 and 4 until the centroids no longer move.

- It can be proved that the k-means algorithm will always terminate, but it does not necessarily find the best set of clusters, corresponding to minimising the value of the objective function.

- The initial selection of centroids can significantly affect the result. To overcome this, the algorithm can be run several times for a given value of k, each time with a different choice of the initial k centroids, the set of clusters with the smallest value of the objective function then being taken.

Initializing Clusters for K-Means: We want to pick points that have a good chance of lying in different clusters.

There are two approaches.

1. Pick points that are as far away from one another as possible.

2. Cluster a sample of the data, perhaps hierarchically, so there are k clusters. Pick a point from each cluster, perhaps that point closest to the centroid of the cluster.